applied sciences

MDPI

Article

Hybrid Data Curation for Imitation Learning with Physics-Generated Trajectories

Mincheol Lee [1], Deun-Sol Cho [2] and Won-Tae Kim [1,2,*]

[1] Future Convergence and Engineering, Computer Science Department, Korea University of Technology and Education, Cheonan-si 31253, Republic of Korea; mc980708@koreatech.ac.kr
[2] Computer Science and Engineering, Computer Science Department, Korea University of Technology and Education, Cheonan-si 31253, Republic of Korea; whemsthf123@koreatech.ac.kr
* Correspondence: wtkim@koreatech.ac.kr

Abstract

Robotic manipulators were initially introduced to replace repetitive human labor and have since evolved to perform complex tasks in dynamic environments. In such systems, imitation learning and reinforcement learning models capable of real-time trajectory generation are widely applied. Among these approaches, imitation learning enables rapid training when high-quality datasets are available. However, it suffers from high costs associated with collecting expert demonstration data and significant performance variability depending on data quality. Recently, learning approaches utilizing large-scale datasets have been explored, but they often struggle to guarantee reliable performance in tasks requiring precise control and incur substantial computational costs for model construction, limiting their applicability as a general-purpose learning strategy. To address these limitations, this paper proposes an imitation learning framework that integrates sampling-based motion planning with a hybrid data curation strategy. The proposed method employs a sampling-based planner (e.g., RRT*) to generate diverse physically feasible trajectories, thereby reducing the cost of acquiring expert demonstration data. The generated trajectories are then curated through clustering-based grouping and rule-based filtering to select high-quality training samples from large-scale datasets. The proposed framework automatically generates physically feasible trajectories while selecting high-quality data from large trajectory pools, thereby improving training stability and reducing data-related costs. Experimental results demonstrate that the proposed method achieves an average success rate of 79.1% (95% CI: 74.3–83.2%) and produces trajectories with shorter trajectories, lower final distances, and reduced joint movements compared to conventional filtering methods.

Keywords: imitation learning; motion planning; clustering

Check for updates

Academic Editor: Samuel Okegbile

Received: 24 February 2026
Revised: 14 March 2026
Accepted: 17 March 2026
Published: 19 March 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

High-degree-of-freedom (DoF) robot manipulators serve as essential components for flexible manipulation in complex environments and play an increasingly critical role in industrial automation and intelligent manufacturing systems [1]. Composed of multiple links and joints, these systems operate in high-dimensional state spaces and must generate physically feasible motions to reach diverse target configurations within constrained workspaces [2]. In such settings, computational efficiency and scalability become fundamental design requirements, particularly when environmental conditions and task specifications change frequently [3].

Traditional motion planning approaches rely on explicitly defined kinematic and dynamic models, employing graph-search- or sampling-based algorithms to generate collision-free trajectories [4]. Although these methods ensure feasibility and constraint satisfaction, their computational complexity increases significantly with increasing DoF and environmental complexity. Furthermore, dynamic environments often require repeated replanning, limiting real-time applicability. The stochastic nature of sampling-based planners may also produce structurally different trajectories for identical start-to-goal pairs, which can reduce execution consistency and complicate downstream learning [5].

To address these limitations, data-driven motion generation methods have emerged as promising alternatives [6]. Neural network-based policies can generate motions directly through inference once trained, bypassing iterative optimization and search [7]. Reinforcement Learning (RL) and Imitation Learning (IL) represent two dominant paradigms for policy learning in high-dimensional robotic control [8]. Although RL learns through interaction with the environment, it typically requires extensive exploration and a high training cost [9]. In contrast, IL learns policies from demonstration data and often provides faster convergence and improved stability when high-quality demonstrations are available [10].

In parallel, Digital Twin (DT) integrated with Internet of Things (IoT) technologies has enabled continuous synchronization between physical robotic systems and their virtual replicas. In DT-enabled pipelines, sensor data from IoT devices support real-time monitoring, predictive simulation, and iterative policy updates within the virtual environment. However, the effectiveness of IL in such frameworks strongly depends on the quality and structural diversity of the trajectory dataset [11]. Even when trajectories are physically feasible, automatically generated motion candidates may contain redundant or task-irrelevant trajectories due to stochastic sampling processes [12]. As dataset size increases, such redundancy can degrade learning efficiency, increase computational cost, and negatively affect generalization performance [13]. Therefore, systematic trajectory curation that preserves structural representativeness while removing redundancy is essential for scalable IL in DT-based robotic systems.

Unlike existing trajectory filtering approaches that primarily focus on human-collected demonstrations or influence-based sample evaluation, this study addresses redundancy arising specifically from physics-based sampling planners. We propose a scalable imitation learning framework that automatically generates feasible motion trajectories using a physics-based planner, eliminating the reliance on expert teleoperation [14]. To manage structural diversity within planner-generated trajectory pools, we introduce a hybrid trajectory filtering strategy that combines clustering-based grouping with rule-based quality assessment. By selectively leveraging trajectory clusters corresponding to specific task conditions and target states, the proposed approach reduces redundant samples while maintaining representative trajectory structures, thereby improving both policy performance and data efficiency. For clarity, we use the term "trajectory filtering" to refer to data curation processes that select informative trajectories from planner-generated datasets.

2. Related Works

Imitation Learning (IL) is a learning paradigm in which a robot acquires a control policy by observing and mimicking human demonstrations [15]. Early studies referred to this approach as Learning from Demonstration (LfD) or Programming by Demonstration, and it was originally proposed as an alternative to reinforcement learning to address its low sample efficiency and long exploration times [16]. Schaal (1996) demonstrated that demonstration data can be leveraged not merely as action replicas but as a means to learn the underlying dynamics of a task, significantly improving learning speed and stability even in nonlinear control problems [17].

Subsequently, imitation learning was extended to a wide range of robotic manipulation tasks using robotic arms, where the quality and quantity of demonstration data were repeatedly shown to have a decisive impact on the final policy performance [18]. Traditional approaches predominantly relied on expert demonstrations collected via teleoperation by skilled human operators. While this method ensures high-quality data, it inherently suffers from scalability limitations due to the substantial human labor and time costs required for large-scale dataset construction and task diversification [19].

To reduce dependence on expert demonstrations, recent studies have explored various strategies for minimizing human involvement in demonstration data collection. For example, Efficient Data Collection for Robotic Manipulation via Compositional Generalization (2024) proposed an in-domain data collection strategy that exploits the compositional generalization capability of policies to cover a broader range of task variations with the same data collection effort [20]. Similarly, Tool-as-Interface: Learning Robot Policies from Observing Human Tool Use (2025) demonstrated that robot policies can be learned without direct robot manipulation by reducing the embodiment gap through Gaussian Splatting-based 3D scene reconstruction and tool-centric action representations derived from videos of human tool use [21].

As the scale of demonstration datasets increases, the inclusion of low-quality demonstrations, redundant trajectories, and task-irrelevant behaviors becomes more likely, thereby degrading learning efficiency and policy performance. Consequently, trajectory filtering techniques that selectively utilize only informative demonstrations from large, pre-collected datasets have emerged as an important research topic. Behavior Retrieval: Few-Shot Imitation Learning by Querying Unlabeled Datasets (2023) demonstrated the effectiveness of post hoc data selection by retrieving task-relevant state–action pairs from unlabeled large-scale datasets [22]. CUPID (2025) applied influence functions to imitation learning to quantitatively estimate each demonstration's contribution to closed-loop policy performance (expected return), enabling the removal of harmful data and the selective curation of beneficial trajectories [23]. SCIZOR (2025) combined self-supervised task progress estimation with similarity-based deduplication to automatically filter suboptimal and redundant state–action pairs, achieving an average performance improvement of 15.4% [24].

Although these studies have made significant progress in reducing demonstration costs and improving learning efficiency in large-scale data regimes, most existing approaches still rely heavily on human expert demonstrations, observation-based video data, or large-scale simulation environments. In particular, the systematic automatic generation of physically feasible trajectories under real-world constraints—and the exploitation of their structural properties for data selection—has not been sufficiently explored. Moreover, simulation-dependent approaches continue to suffer from limited real-world transfer due to the persistent sim-to-real gap [25]. These limitations highlight the need for scalable methods that can automatically construct and curate trajectory datasets from physics-generated motion planning results.

The novelty of this work lies in the integration of physics-generated trajectory datasets with a hybrid data curation pipeline that combines task-conditioned clustering and rule-based trajectory quality evaluation. Unlike conventional imitation learning pipelines that rely on human demonstrations or large-scale vision datasets, the proposed framework focuses on scalable trajectory filtering for planner-generated motion data. This perspective shifts the focus from policy architecture design to scalable trajectory data curation for physics-generated motion datasets.

To address these limitations, this work makes the following contributions. First, we propose a low-cost and scalable data construction framework that automatically generates physically feasible trajectory data using a physics-based motion planner, without requiring

human expert demonstrations. Second, we introduce a hybrid data selection method that combines clustering-based grouping with rule-based quality evaluation. Third, by selectively utilizing trajectory clusters corresponding to specific task conditions and goal states, the proposed approach effectively filters out unnecessary data, thereby improving both the performance of imitation learning policies and the efficiency of computational resource utilization.

3. Background

3.1. Imitation Learning

Imitation Learning (IL) is a methodology for learning a policy by mimicking expert demonstrations without explicitly defining a reward function [26]. Representative approaches include Behavior Cloning (BC) and Inverse Reinforcement Learning (IRL) [27]. BC directly learns state–action pairs, making it simple and efficient to implement. However, when the distribution of expert data differs from the distribution encountered during actual policy execution, covariate shift occurs, leading to compounding errors. Additionally, collecting expert demonstration data requires significant costs, posing practical financial challenges for applying imitation learning. In contrast, IRL infers the latent reward function implicitly followed by the expert and then learns the policy through reinforcement learning (RL) [28]. While it offers high expressiveness (enabling more nuanced policies via reward inference), it suffers from increased computational costs.

Recent research has actively progressed to overcome these limitations of IL. For example, data re-collection methods like DAgger (Dataset Aggregation) mitigate distribution shift problems [29], while adversarial IL techniques such as GAIL (Generative Adversarial Imitation Learning) [30] and AIRL (Adversarial Inverse Reinforcement Learning) [31] enable more accurate learning of expert policy structures. Furthermore, Vision–Language–Action (VLA) policy models utilizing large-scale Multi-Embodiment robot datasets, such as RT-1, RT-2, and Open-X Embodiment, significantly improve generalization performance and expand real-world applicability. In particular, RT-2 fuses vision, language, and action through transformer architectures to achieve zero-shot generalization in dynamic environments, but high-quality trajectory filtering remains essential to mitigate distribution shifts. Nevertheless, IL still faces structural and ethical limitations, including the high cost of acquiring high-quality demonstration data, difficulties in generalization to new environments, policy stability sensitive to data quality, and amplification of expert biases during real-world deployment.

3.2. Behavioral Cloning

Behavioral Cloning (BC) is a method that learns a deterministic policy to predict the same action as the expert given a state from expert demonstrations. BC has advantages in its simple structure, fast training process, and feasibility with low computational resources. However, when encountering new states not present in the expert data, the policy can rapidly accumulate errors due to distribution shift, which is a major limitation of BC in real-world robot control.

The training objective of BC is to minimize the following loss function:

$$\mathcal{L}_{\mathrm{BC}} = \mathbb{E}_{(\mathbf{s},\mathbf{a})\sim\mathcal{D}}\left[\|\mathbf{a} - \pi_\theta(\mathbf{s})\|^2\right]$$

$\mathcal{D}$ is the expert demonstration dataset, and π_θ is the policy network.

In this study, BC is used as the baseline model, employing an MLP (Multi-Layer Perceptron)-based BC MLP structure that takes state features as input and directly outputs the next-step joint values. The feedforward structure of MLP is suitable for deterministic

mapping in joint-space control and avoids complex sequence dependencies, making it an appropriate baseline for comparing the effectiveness of data selection techniques. The hybrid filtering framework proposed in this study is expected to contribute to alleviating BC's covariate shift problem.

3.3. Recurrent Neural Network

Recurrent Neural Networks (RNNs), specialized for processing sequential data, can model temporal dependencies and are widely used in problems where time-series information is critical, such as continuous joint states or trajectories in robots. RNN reflect past information in current action predictions through hidden states, showing effective characteristics in control problems where different actions are required over time even for the same input. The BC RNN model applies this RNN structure to Behavior Cloning to overcome the per-timestep independent prediction limitation of MLP-based BC. Since actual robot trajectories exhibit long-term dependencies, structures like LSTM (Long Short-Term Memory) or GRU (Gated Recurrent Unit) are known to learn more stably than vanilla RNN. GRU reduces parameters by approximately 25% compared to LSTM while maintaining similar performance [32]. BC RNN reflects the temporal continuity of trajectories, enabling smoother and more stable prediction of entire trajectories. However, for very long sequences, recent studies indicate that Transformer-based models (e.g., Decision Transformer) excel in handling long-term dependencies ([33]; robotics applications in 2025).

3.4. Dropout

Dropout is a representative regularization technique designed to prevent overfitting in neural networks by probabilistically deactivating a subset of neurons during training, thereby discouraging excessive reliance on specific features. In imitation learning settings with diverse trajectory patterns and noisy demonstrations, Dropout facilitates more generalized policy learning, typically yielding a 10–15% improvement in generalization performance [34].

When applying Dropout to recurrent neural networks (RNNs), such as LSTM or GRU, standard fully connected Dropout is generally avoided due to its disruptive effect on recurrent state dynamics. Instead, recurrent Dropout or variational Dropout is commonly employed [35]. In particular, variational Dropout applies a consistent dropout mask across timesteps, preserving temporal consistency in sequence modeling. In this study, the $BC_{\text{RNN-Dropout}}$ model incorporates these RNN-specific Dropout strategies, with dropout rates typically ranging from 0.2 to 0.5, to reduce overfitting in RNN-based behavioral cloning and enable more stable policy learning under significant data quality variations.

3.5. Mixture Density Network

Mixture Density Networks (MDNs) extend conventional neural network architectures by modeling probabilistic output distributions rather than producing single-point predictions [36]. An MDN outputs the mixing coefficients (π), means (μ), and variances (σ) of K Gaussian mixture components, thereby representing multimodal action distributions.

The $BC_{\text{RNN-MDN}}$ model combines RNN-based temporal modeling with MDN-based probabilistic action prediction, allowing multiple plausible actions to be inferred for the same state. This formulation is particularly advantageous in robot manipulation scenarios where multiple valid trajectories can achieve an identical goal. The MDN training objective is defined as:

$$\mathcal{L}_{\text{MDN}} = -\sum_{t=1}^{T} \log\left(\sum_{k=1}^{K} \pi_k(t)\, \mathcal{N}(a_t \mid \mu_k(t), \sigma_k(t))\right)$$

Despite its expressive power, MDN training is known to suffer from challenges such as instability, slow convergence, and mode collapse, with performance highly sensitive to data quality, dataset size, and filtering strategies. Effective training often requires careful parameter initialization and temperature annealing, and several stabilization techniques have been proposed in recent studies. In the context of this work, MDNs are particularly well suited for capturing multimodal optima inherent in trajectories generated by physics-based sampling motion planners.

3.6. Random Sampling Algorithm

Random sampling is a core technique in sampling-based motion planning, enabling efficient exploration of high-dimensional configuration spaces without exhaustive search. Representative algorithms such as RRT (Rapidly exploring Random Tree) and PRM (Probabilistic Roadmap) incrementally construct feasible robot trajectories by sampling random states and connecting collision-free configurations. Due to their probabilistic completeness, these methods can find valid solutions with high probability as the number of samples increases.

In simulation environments such as Gazebo, sampling-based planners generate physically feasible trajectories while considering dynamic constraints. However, they often suffer from low sample efficiency in complex environments, particularly in narrow passages. These characteristics motivate the need for trajectory filtering strategies that can select informative samples from large sets of planner-generated trajectories. Such random sampling-based data generation forms the foundation of the hybrid trajectory filtering framework proposed in this study.

4. Hybrid Data Curation for Imitation Learning with Physics-Generated Trajectory Framework

4.1. Data Generation Module

The data generation module constitutes the first stage of the proposed framework and is responsible for generating control trajectory data from the robot manipulator's initial configuration to the target configuration using a physics-based motion planner. As shown in Figure 1, it consists of two main components: a Physics-based Motion Planner and a hybrid trajectory filter that selects high-quality trajectory candidates from the generated data.

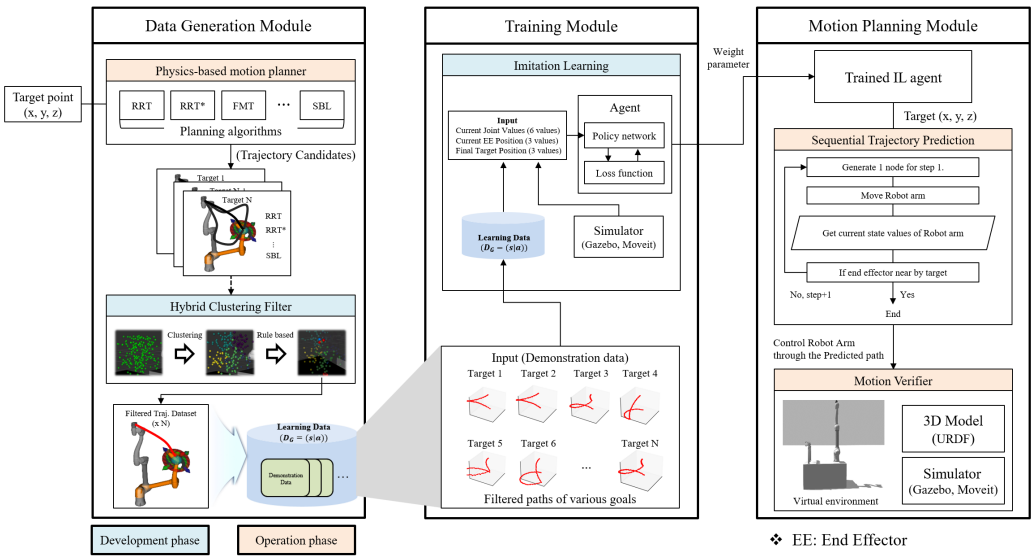

Figure 1. Overview of the proposed imitation learning framework using physics-generated trajectories and hybrid data clustering.

The motion planner employs 23 random sampling-based algorithms, including RRT, RRT*, and FMT*, to generate diverse trajectory candidates even for identical start and goal configurations. Since sampling-based planners can produce trajectories that deviate from the intended motion behavior of human engineers, particularly in complex environments, a data filtering process is essential to ensure the reliability and performance of imitation learning models. To address this issue, the proposed hybrid trajectory filter operates in a two-stage manner to select high-quality training data.

4.1.1. Physics-Based Motion Planner

A physics-based motion planner is an algorithmic system that generates collision-free trajectories enabling a robot to safely and efficiently reach a target position while satisfying various physical and operational constraints in complex environments. Traditional motion planning methods can be broadly categorized into sampling-based approaches (e.g., RRT*, PRM; see Table 1), optimization-based approaches (e.g., model predictive control), and graph-search-based approaches (e.g., A*). Each category exhibits distinct trade-offs in terms of solution optimality, computational efficiency, and scalability.

Recent studies have actively explored hybrid approaches that integrate data-driven or learning-based components with classical motion planning techniques to improve real-time adaptability, robustness to uncertainty, and generalization performance in high-dimensional and dynamic environments. Motion planning remains a core technology in applications such as autonomous driving, industrial robotics, and medical robotics. Nevertheless, significant challenges persist, including scalability to large-scale problems, sim-to-real transfer, and the assurance of safety in unpredictable scenarios.

Table 1. 23 Motion Planning Algorithms.

23 Motion Planning Algorithms					
RRT	RRT*	RRTConnect	TRRT	PRM	PRM*
KPIECE	BKPIECE	LBKPIECE	FMT	BFMT	PDST
STRIDE	BiTRRT	LBTRRT	BiEST	ProjEST	LazyPRM
LazyPRM*	SPARS	SPARStwo	EST	SBL	

4.1.2. Clustering

Applying a purely rule-based filtering strategy may introduce subjective bias, as the selection criteria can heavily depend on the engineer's prior assumptions and preferences. To mitigate this issue, the proposed framework first performs clustering to objectively group data according to intrinsic patterns, followed by a secondary rule-based filtering stage applied within each cluster. This two-stage process reduces subjectivity while ensuring data quality.

In the first selection stage, the entire pool of collected trajectory data is grouped using clustering techniques. The goal of this clustering stage is not to discover complex trajectory efficiency and motion quality but to organize trajectories according to task-relevant goal conditions. Since the trajectories are generated by sampling-based motion planners with identical start configurations, the final end-effector position serves as a reliable descriptor for grouping task-conditioned trajectory sets.

In this study, representative clustering algorithms are employed to group trajectories before the subsequent rule-based evaluation stage. To evaluate the general applicability of the proposed framework, commonly used clustering algorithms such as K-Means, DBSCAN, and Hierarchical clustering are implemented in the experimental study.

Clustering is performed based on the final EE target position of each robot control trajectory. Under the considered motion planning setting, trajectories are generated under iden-

tical start configurations and identical goal specifications. Therefore, the primary variation among trajectories typically arises from differences in trajectory efficiency—such as detours, curvature, or redundant motion—rather than fundamentally different trajectory structures.

While sequence-alignment methods such as Dynamic Time Warping (DTW) were considered for capturing trajectory efficiency and motion quality, comparative evaluations demonstrated that EE position-based clustering yields superior policy success rates and training stability for large-scale physics-generated datasets.

This approach alleviates the problem of engineer-dependent filtering criteria by enforcing objective, data-driven grouping in the initial stage. Applying a secondary rule-based filter within the pre-selected clusters further guarantees a baseline level of data quality by selecting trajectories that exhibit higher motion efficiency and stability.

The key characteristics of the three clustering methods used in this study are summarized in Table 2. K-Means represents centroid-based clustering, DBSCAN corresponds to density-based clustering, and Hierarchical clustering is distance-based. Comparative experiments are conducted to identify the clustering strategy that best captures task-conditioned trajectory distributions within the generated dataset. Although clustering hyperparameters such as the number of clusters in K-Means or the neighborhood radius in DBSCAN can influence grouping behavior, the objective of this study is to evaluate the trajectory filtering framework rather than to optimize individual clustering algorithms. In practice, these parameters can be adjusted depending on task complexity and dataset scale.

Table 2. Comparison of clustering methods used in this study.

Criterion	K-Means	DBSCAN	Hierarchical
Clustering basis	Centroid-based	Density-based	Distance-based merging
Hyperparameters	K (number of clusters)	ε (radius), MinPts	K (number of clusters)
Cluster shape	Spherical/convex	Arbitrary	Arbitrary
Noise handling	Poor	Strong	Poor

4.1.3. Rule-Based Filter

To transcend the limitations of initial EE-based grouping, the trajectory data are further refined through a secondary rule-based filtering process. This stage serves as a structural optimization process that captures the morphological efficiency and temporal stability of the trajectories. This step aims to guide the learning model toward more efficient and stable trajectories. Three key metrics are employed for this secondary selection, as described below.

To aggregate these metrics into a unified evaluation criterion, a composite score is computed by combining Total Joint Movement, Smoothness, and Trajectory Length with equal weighting. This design choice avoids implicitly prioritizing a specific motion characteristic and provides a neutral baseline for trajectory quality evaluation, since the relative importance of these metrics may vary depending on the task context. Nevertheless, the proposed framework remains flexible and can incorporate task-dependent weighting schemes in future extensions.

4.1.4. Total Joint Movement

Total Joint Movement quantifies the cumulative amount of joint motion required to execute a trajectory. It is computed as the sum of absolute joint angle changes across all joints and time steps.

$$M = \sum_{i=1}^{N} \sum_{j=1}^{J} |\theta_{i,j} - \theta_{i-1,j}|$$

where:

- N is the total number of time steps in the trajectory;
- J is the number of joints (e.g., $J = 6$ for a 6-DoF robot);
- $\theta_{i,j}$ denotes the angular position of the j-th joint at the i-th time step.

A smaller value of Total Joint Movement indicates a more energy-efficient trajectory with reduced mechanical wear.

4.1.5. Smoothness

Smoothness measures how smoothly a trajectory is executed by evaluating variations in acceleration. Lower smoothness values correspond to stable robot motions without abrupt changes, which is crucial for reducing vibration and improving precision in real-world physical systems.

$$S = \sum_{t=1}^{n-2} \left(a_x(t)^2 + a_y(t)^2 + a_z(t)^2 \right)$$

where:

- $a_x(t)$, $a_y(t)$, and $a_z(t)$ denote accelerations along the x, y, and z axes at time step t, respectively.

4.1.6. Trajectory Length

Trajectory Length represents the total distance traveled by the robot's end-effector in three-dimensional space. It is defined as the cumulative Euclidean distance between consecutive end-effector positions along a trajectory. A trajectory whose length is closer to the shortest trajectory between the start and target positions is considered more efficient.

$$L = \sum_{i=1}^{N-1} \|\mathbf{P}_i - \mathbf{P}_{i-1}\|_2$$

where:

- $\mathbf{P}_i$ denotes the 3D position (x, y, z) of the end-effector at the i-th time step;
- N is the total number of time steps in the trajectory.

For each trajectory candidate, the average values of the three evaluation metrics—Total Joint Movement, Smoothness, and Trajectory Length—are computed and aggregated into a single composite score via simple summation. Based on this score, trajectories are ranked and categorized accordingly: the top N trajectories with the lowest scores are selected as high-quality (high-efficiency) trajectories, while the top N trajectories with the highest scores are identified as low-quality (low-efficiency) trajectories.

These metrics serve as the core criteria for quality assessment and ranking of trajectory candidates. Through this filtering process, noise within the training dataset is reduced while high-quality data are retained, ultimately lowering learning costs and improving training efficiency.

4.2. Training Module

Learning Data

The learning data consist of trajectory data associated with a target object. Each trajectory includes the three-dimensional position of the target with respect to the end-effector, as well as step-wise joint angle information describing the joint rotations from the start position to the target position.

Depending on the experimental setting, 1, 5, or 20 trajectories per target were selected and used for training. The selection process ranked trajectory candidates based on three

evaluation criteria: total joint movement, trajectory smoothness, and total trajectory length. Trajectories were selected in descending order of efficiency according to these metrics.

In this context, high-quality data refer to highly efficient trajectories that minimize joint movement, exhibit smooth motion, and follow short trajectories, whereas low-quality data correspond to inefficient trajectories that exhibit larger joint motions, reduced smoothness, and longer trajectories.

4.3. Motion Prediction Module

This module predicts the sequence of nodes that constitute a trajectory from the initial state to the target state. The values to be predicted are the joint rotation states of each step, corresponding to the six robot joints, denoted as Joint 1 through Joint 6. The trajectory prediction module consists of a trained imitation learning (IL) model, a robot model in a simulation environment, and a motion planner.

4.3.1. Motion Verifier

The trained IL model predicts a trajectory from the current state to the target state when the engineer provides a target specification. In this study, a trajectory is represented as a sequence of nodes, each containing the state values of the six joints. Before deployment in real-world scenarios, the predicted trajectory undergoes a verification process performed by the Motion Verifier.

The Motion Verifier is composed of a robot model (3D model) and a motion planning simulator in a virtual environment. It validates the predicted trajectory by checking, at each step, whether the node corresponds to a physically feasible robot configuration and whether any collisions with obstacles occur during execution.

4.3.2. Trained IL Agent

In this study, multiple IL models—namely BC MLP, BC RNN, BC RNN Dropout, and BC RNN MDN—were trained using the filtered trajectory data and employed as policy models for predicting the next-step control commands of the robotic manipulator.

The trained IL agent takes the current state as input and outputs the joint values for the next time step, thereby generating the entire trajectory sequentially. By leveraging IL-based models, trajectories can be generated rapidly using only the learned policy, without relying on a motion planner. This approach enables fast adaptation to new situations within a short time frame, without requiring iterative online optimization or sampling-based search.

5. Experiment Setting

5.1. Experimental Environment and Data Generation

The experiments were conducted in a Gazebo simulator under the ROS (Robot Operating System) environment. The virtual robot model in Gazebo was described using URDF (Unified Robot Description Format), and MoveIt was adopted as the motion planner for robot control.

For trajectory data generation, motion planning algorithms provided by OMPL (Open Motion Planning Library), including RRT, RRT*, and FMT, were utilized. For each target, the motion planner was executed three times per algorithm, resulting in a total of 69 diverse trajectory candidates per target. The collected trajectory candidates were subsequently refined using the proposed HC Filter and then used as training data.

The robot employed in this study was the Doosan Robotics H2017 model. As shown in Table 3, which compares robot model specifications with those used in prior studies, the H2017 has a higher payload capacity and a wider working range than the robots adopted in previous research.

Table 3. Comparison of Robot Specifications with Previous Studies.

Reference	Robot Model	Payload	Reach	Task Range
One-Shot Visual Imitation Learning via Meta-Learning [37]	PR2	∼2.2 kg	∼850 mm	0.6 m
One-Shot Imitation Learning [38]	Fetch	4.5 kg	∼850 mm	∼0.6–1 m
One-Shot Imitation Learning [38]	Sawyer	4 kg	∼1260 mm	∼0.6–1 m
Behavior Retrieval: Few-Shot Imitation Learning by Querying Unlabeled Datasets. [22]	Franka Panda	3 kg	∼850 mm	∼0.5–1 m
Our Paper	**Doosan H2017**	**20 kg**	**1700 mm**	**0.5 m**

5.2. Training and Evaluation Settings

The learning model incorporated four variations based on Behavior Cloning (BC): BC MLP, BC RNN, BC RNN Dropout, and BC RNN MDN. For trajectory prediction, a Sequential Approach was adopted. Instead of predicting the entire trajectory simultaneously, this method iteratively predicts the state of the next step based on the current state input until the target point is reached. This sequential approach offers distinct advantages, including high memory efficiency and the capability to respond in real time to dynamic environmental changes.

As defined in Table 4, the success criterion for this experiment was established as a Euclidean distance of less than 0.05 m (5 cm) between the robot end-effector's final position and the target point. This threshold of 5 cm aligns with the success criteria employed in prior studies addressing similar robotic reaching tasks.

Table 4. Comparison of Success Criteria in Prior Research on Robotic Arm Control.

Reference	Task Success Criterion	Task Range	Robot DoF
One-Shot Visual Imitation Learning via Meta-Learning [37]	Reaching the target point within 0.05 m (5 cm)	0.6 m × 0.6 m	7-DoF
One-Shot Imitation Learning [38]	Success rate of reaching within 5 cm or 10 cm	Approx. 0.6–1 m (Fetch/Sawyer experiments)	7-DoF
Behavior Retrieval: Few-Shot Imitation Learning by Querying Unlabeled Datasets [22]	Reaching within approx. 5 cm, per-task policy success rate	Approx. 0.5–1 m (Franka, etc.)	7-DoF
Reinforcement Learning for Solving Robotic Reaching Tasks in the Neurorobotics Platform [39]	Minimum proximity success within 5 cm used as success criterion	Approx. 1 m within Neurorobotics Platform	6-DoF

6. Experiment Result

This chapter presents the experimental results conducted to validate the effectiveness of the proposed HC Filter. The comparative analysis focused on the average target reach success rate and the quality of generated trajectories, depending on the application of data selection techniques.

6.1. Comparison of Success Rates by Data Selection Method

To evaluate the performance of the proposed methodology, the General model (without data selection), the Rule-based model, and existing trajectory filtering studies (BehaviorRetrieval, SCIZOR, CUPID) were established as baselines. Four variations based on Behavior Cloning (BC) were utilized as learning models: BC MLP, BC RNN, BC RNN Dropout, and BC RNN MDN. The success criterion was defined as a distance of less than 0.05 m (5 cm) between the end-effector and the target point.

It should be noted that the primary objective of this study is not to propose a new policy architecture, but to investigate trajectory data curation strategies for imitation learning using physics-generated motion planning data. Therefore, the selected baselines were

chosen as representative trajectory filtering and data selection approaches that have been widely explored in imitation learning research. While some of these methods are often applied to high-dimensional or vision-based datasets, they remain relevant benchmarks for evaluating the effectiveness of data selection strategies.

To ensure fair comparison, all filtering methods were evaluated using the same imitation learning models and identical experimental settings. In this way, the comparison focuses on the impact of trajectory data selection rather than differences in policy architectures, allowing the evaluation to isolate the effect of filtering strategies on policy performance.

Unlike prior trajectory filtering approaches such as Behavior Retrieval, SCIZOR, and CUPID, which primarily rely on similarity-based retrieval or influence estimation for selecting training samples, the proposed framework emphasizes computational scalability for planner-generated trajectory datasets. In particular, the hybrid filtering strategy operates directly on motion planning outputs through task-conditioned clustering and rule-based trajectory quality evaluation, enabling efficient grouping and ranking without requiring expensive influence estimation or pairwise similarity computations.

As shown in the experimental results (Figure 2), the proposed clustering-based filtering techniques (K-Means, DBSCAN, Hierarchical) and their hybrid variants combined with rule-based methods achieved higher success rates compared to all baseline models. Specifically, the K-Means (Hybrid) technique demonstrated the highest success rate of 79.1%, while the Hierarchical (Hybrid) and DBSCAN (Hybrid) techniques achieved 78.4% and 76.6%, respectively. When compared to the General model without data selection (54.1%) and the Rule-Based General model (40.0%), these results indicate that the proposed methodology effectively enhances the quality of training data, thereby significantly improving policy performance.

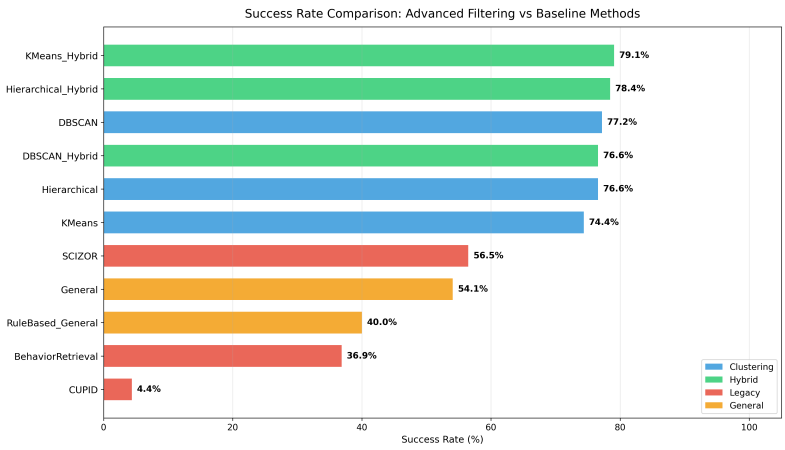

Figure 2. Comparison of Average Success Rates by Data Selection Method.

Furthermore, in the case of the K-Means technique, applying clustering alone (Pure) yielded a success rate of 74.4%. However, the Hybrid approach, which incorporates the rule-based filter, reached 79.1%, showing an additional performance improvement of 4.7 percentage points. This demonstrates that the secondary rule-based filtering, following the primary data selection via clustering, provides more precise control over data quality.

6.2. Comparison of Generated Trajectory Quality

To evaluate the qualitative efficiency of the generated trajectories in addition to the success rate, we conducted a comparative analysis of (1) Average Final Distance (Goal Reaching Accuracy), (2) Average Trajectory Length (Trajectory Efficiency), and (3) Average Joint Movement (Actuation Efficiency).

As illustrated in Figure 3, the proposed clustering and hybrid methods demonstrated superior performance compared to the baseline models across all trajectory quality metrics.

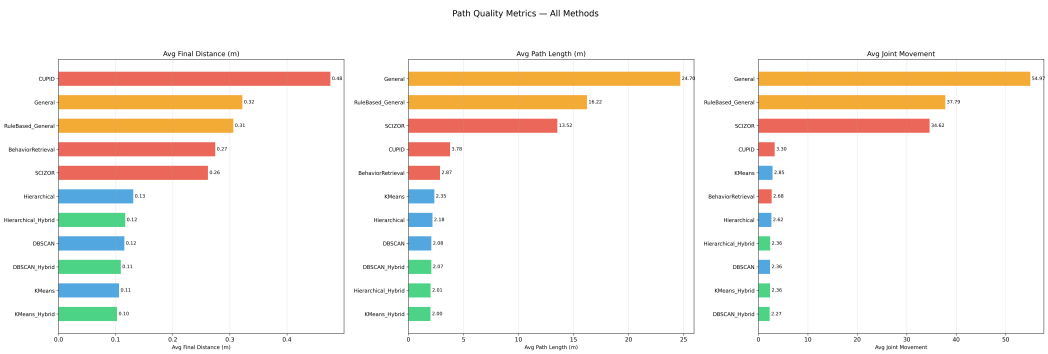

Figure 3. Comparison of Trajectory Quality Metrics (Distance, Length, and Joint Movement).

Goal Reaching Accuracy (Final Distance): The proposed techniques (e.g., K-Means Hybrid: 0.103 m) achieved significantly closer proximity to the target at the final position compared to the baselines (e.g., General: >0.3 m).

Trajectory Efficiency (Trajectory Length): Whereas the baseline models generated inefficiently long trajectories (e.g., General: >25 m), the proposed methods produced highly concise and efficient trajectories (e.g., K-Means Hybrid: approx. 2.0 m).

Actuation Efficiency (Joint Movement): The proposed techniques (e.g., K-Means Hybrid: approx. 2.36) drastically reduced the total joint movement compared to the baselines (e.g., General: >50). This confirms that the proposed methods generate efficient trajectories, which are beneficial in terms of energy consumption and mechanical wear.

In conclusion, the proposed hybrid data filtering technique was demonstrated to be highly effective not only in enhancing the success rate of imitation learning but also in generating high-quality trajectories that ensure both the accuracy and efficiency essential for real-world robot operations.

6.3. Comparison of Success Rates by Imitation Learning Algorithm

This section validates whether the proposed hybrid data clustering filter consistently enhances performance across a broad spectrum of imitation learning algorithms, demonstrating its generalizability beyond specific model architectures.

The experiments measured success rates by applying the proposed selection techniques (K-Means, DBSCAN, Hierarchical, and their hybrid variants) as well as the baselines (General, SCIZOR, etc.) to each of the four models defined in Section 5.2 (BC MLP, BC RNN, BC RNN Dropout, and BC RNN MDN).

The most notable results were observed in models based on Recurrent Neural Networks (RNN), as shown in Figure 4. When integrated with the BC RNN model, the proposed clustering techniques (DBSCAN Hybrid: 96.2%, Hierarchical Hybrid: 96.2%) achieved significantly higher success rates compared to the General model (82.5%). Specifically, for the BC RNN Dropout model, the proposed K-Means (Hybrid) and K-Means (Pure) techniques recorded a success rate of 97.5%. This result can be interpreted as a synergistic effect arising from the RNN effectively learning the sequential nature of trajectory data, Dropout mitigating overfitting, and the hybrid filter providing high-quality training data. The validity of these techniques was also confirmed in the BC MLP model, where Hierarchical (75.0%) and DBSCAN Hybrid (73.8%) methods consistently outperformed the General model (56.2%). This indicates that the proposed filter intrinsically enhances data quality independent of model complexity.

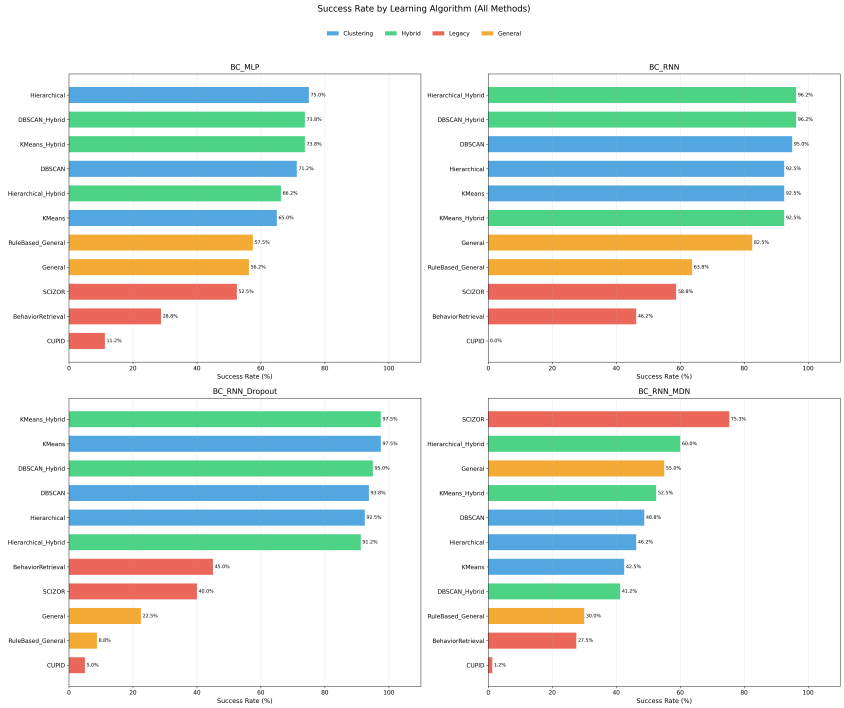

Figure 4. Success Rates of Data Selection Methods by Imitation Learning Algorithm.

However, the BC RNN MDN model exhibited a divergent trend, with the baseline SCIZOR achieving the highest performance at 75.3%. This performance degradation in MDN-based models suggests a fundamental mismatch between the architecture and the filtered data. While MDN is designed to capture complex, multi-modal action distributions, the proposed hybrid filter selectively retains only the most efficient trajectories, effectively transforming a multi-modal planning problem into a unimodal one. This reduction in structural diversity likely hindered the training stability of the MDN, as its expressive capacity for diversity conflicted with the focused optimality of the curated dataset.

This observation highlights a potential trade-off between trajectory efficiency and structural diversity in curated datasets. While the proposed filtering strategy improves learning stability for deterministic policy models such as BC MLP and BC RNN, probabilistic architectures like MDNs may benefit from retaining a broader distribution of feasible trajectories. Future research could explore adaptive filtering strategies that preserve a controlled level of trajectory diversity while removing clearly inefficient trajectories. Such approaches may better support multi-modal policy architectures while maintaining the advantages of trajectory quality filtering.

In conclusion, the proposed hybrid data clustering filter induced a substantial performance enhancement across most imitation learning algorithms—including BC MLP, BC RNN, and BC RNN Dropout—by providing high-quality, streamlined training data.

6.4. Analysis of Success Rates by Selection Technique Across Target Distance Ranges

This section analyzes the impact of variations in the distance from the robot's starting point to the target (Distance Range: 0.0 cm–5.0 cm) on the success rates of each model. The objective is to verify the stability of the control performance maintained by the proposed method despite varying task difficulties, as shown in Figure 5.

1. Stable Dominance in Short- and Medium-Range Tasks (0.0 cm–3.0 cm)
   Maintenance of Top-Tier Performance by Hybrid Techniques: In environments where the target distance was 3.0 cm or less, K-Means Hybrid, DBSCAN Hybrid, and

Hierarchical Hybrid consistently maintained high success rates, often exceeding 80% in short- and medium-range tasks.

Resilience to Increasing Difficulty: Notably, as the distance increased from 0.0 cm to 3.0 cm, the success rate of the General (unselected) model tended to decline. In contrast, the proposed hybrid techniques maintained high success rates based on curated high-quality data, demonstrating robustness against distance variations.

2. Analysis of Performance Inversion in Long-Range Tasks (4.0 cm–5.0 cm)

   Relative Strength of SCIZOR: In the 5.0 cm range, representing the highest task difficulty, the legacy technique SCIZOR recorded the highest performance, with a success rate of approximately 68%.

   Limitations and Implications of Hybrid Techniques: A slight decline in the success rates of the proposed clustering-based techniques was observed as the distance increased. This suggests that trajectory complexity increases with target distance, and filtering based solely on efficiency (e.g., distance and smoothness) may not sufficiently capture the diverse state–action pairs required for reaching distant targets. Nevertheless, hybrid techniques often maintained competitive or higher performance compared to the General model (approx. 50%) and the Rule-Based General model, thereby demonstrating the effectiveness of data selection.

3. Performance Disparity Among Trajectory Filtering Techniques

   Limitations of CUPID: Across all distance ranges, CUPID recorded the lowest success rates (below 10%), suggesting limited compatibility with planner-generated trajectory data in this experimental setting.

   Variability of the Optimal Technique: While the optimal hybrid algorithm (K-Means, DBSCAN, or Hierarchical) varied slightly depending on the distance, all consistently exhibited significantly higher performance compared to the unselected method (see Appendix A for additional results)

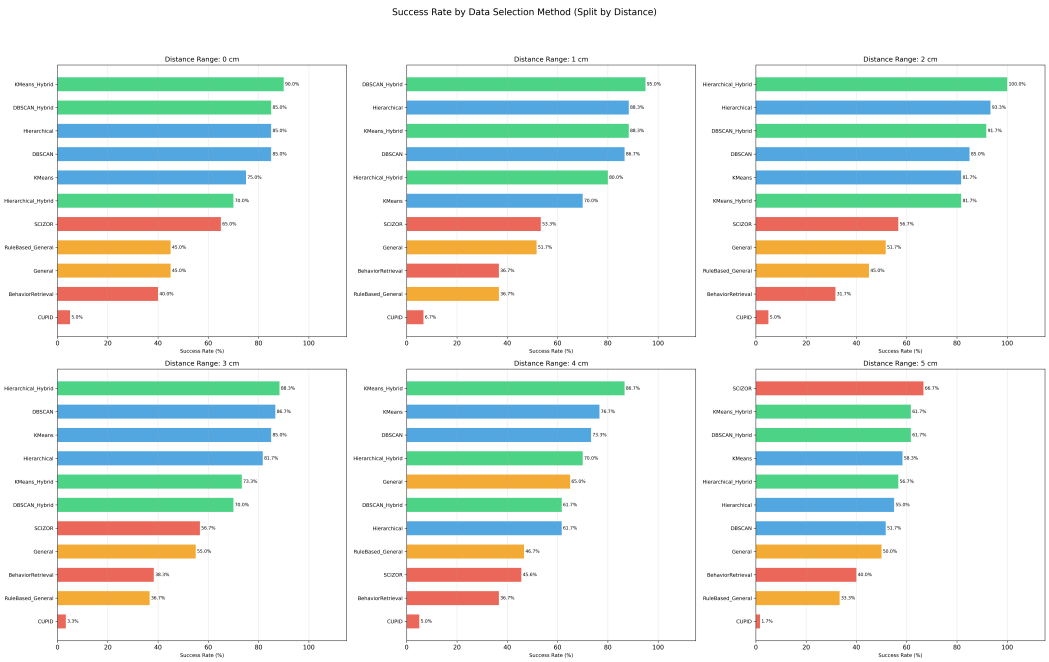

Figure 5. Success Rates by Selection Technique Across Target Distance Ranges.

7. Conclusions

This paper proposes a hybrid data curation framework for imitation learning that leverages physics-generated trajectories to address the high costs associated with expert demonstration collection. The framework automatically accumulates a large-scale set of

candidate trajectories via random sampling algorithms utilizing formula-based motion planners, and subsequently curates high-quality training data through a Hybrid Data Clustering Filter that integrates clustering algorithms (KMeans, DBSCAN, Hierarchical) with rule-based selection. This approach focuses on overcoming the limitations of conventional imitation learning and enhancing practicality in real-world robot control.

Experimental results demonstrated that the model utilizing the proposed technique achieved a target reach success rate of 79.1%, outperforming models without data selection (General) and those applying legacy filtering methods (Cupid, Scizor). This represents a 25.0 percentage point improvement over the General model (54.1%), verifying that enhanced data quality significantly improves the stability and generalization capabilities of the learning policy. Notably, the Hybrid technique significantly reduced the data selection time compared to legacy methods, confirming its effectiveness in improving preprocessing efficiency and reducing system operational costs.

The primary contributions of this study are as follows: First, the proposed trajectory generation framework reduces the costs of imitation learning data collection and is characterized by securing higher-quality training data through filtering. It overcomes the limitations of expert demonstrations by collecting massive amounts of data at low cost and improving learning performance through quality curation. Second, by selectively utilizing training data from clusters corresponding to the target point, the approach conserves computational resources while achieving higher success rates compared to non-selective models. Third, the incorporation of rule-based techniques following clustering further enhances the quality of the generated trajectories.

Future research plans to validate the sim-to-real gap by applying the proposed framework to actual robot hardware and to generalize it to complex scenarios such as dynamic obstacles or multi-robot environments. Additionally, surrogate-assisted optimization techniques could be explored to reduce the computational cost associated with large-scale trajectory generation in simulation environments [40]. Such approaches may improve the efficiency of trajectory dataset construction while maintaining the fidelity of physics-based motion planning. Furthermore, future work will investigate scalable trajectory filtering strategies for large trajectory pools generated in simulation environments, where efficient dataset curation becomes increasingly important as task complexity and planning dimensionality grow.

Author Contributions: Conceptualization, M.L.; methodology, M.L.; software, M.L.; data curation, M.L.; validation, D.-S.C.; formal analysis, D.-S.C.; visualization, D.-S.C.; writing—original draft preparation, D.-S.C.; writing—review and editing, D.-S.C. and W.-T.K.; supervision, W.-T.K.; project administration, W.-T.K.; funding acquisition, W.-T.K. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported in part by the Technology Innovation Program (Development of SDF-Based AI Autonomous Manufacturing Core Technology to Advance the Automobile Industry) funded by the Ministry of Trade, Industry and Energy (MOTIE), Republic of Korea, under Grant RS-2024-00507388 and Korea Evaluation Institute of Industrial Technology (KEIT) grant funded by the Korea government (MOTIE) (RS-2025-02307650, Development and Practice of an On-device AI Functionality and Performance Testing Framework based on NPU).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors upon reasonable request.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Additional Experimental Details and Statistical Analysis

To strengthen the validity of the findings reported in Sections 6.1–6.4 a comprehensive statistical analysis was conducted on the key metrics. This analysis includes 95% confidence intervals (CIs) for success rates (calculated using Wilson's score interval for proportions) and continuous metrics (calculated using t-intervals). Hypothesis testing, including z-tests for proportions and *t*-tests for means, was performed against the General baseline to determine the significance of the improvements. Statistical significance is denoted as follows: *** ($p < 0.001$), ** ($p < 0.01$), * ($p < 0.05$), and ns (not significant). The analysis confirms that the proposed trajectory filtering methods show statistically significant improvements in most performance categories.

Appendix A.1. Success Rate Analysis

Table A1 provides the overall success rates with their corresponding 95% CIs and z-test results. Figure A1 visualizes these results, highlighting the performance gap between the proposed methods and legacy baselines.

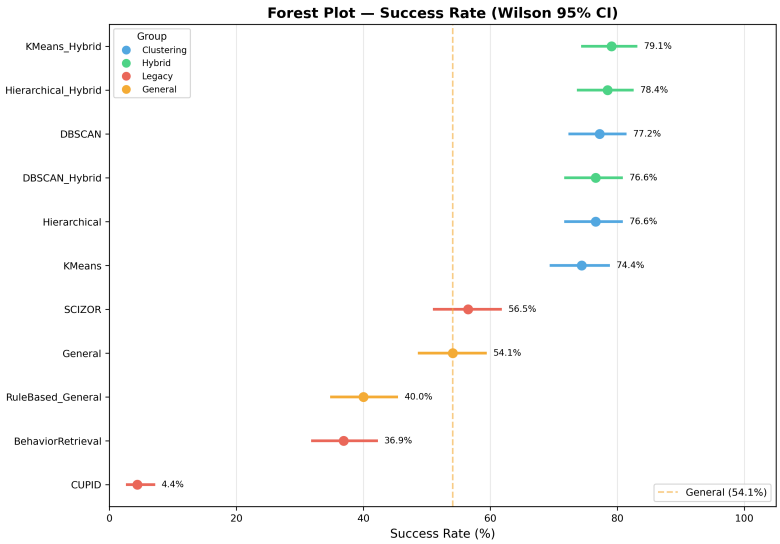

Figure A1. Forest plot of success rates with 95% CI (Wilson method) vs. General baseline (dashed line).

Table A1. Success rate with 95% CI and z-test vs. General baseline.

Data Selection	Group	n	Successes	SR (%)	95% CI	z-stat	*p*-Value
KMeans_Hybrid	Hybrid	320	253	79.1	[74.3, 83.2]	+6.70	$p < 0.001$
Hierarchical_Hybrid	Hybrid	320	251	78.4	[73.6, 82.6]	+6.52	$p < 0.001$
DBSCAN	Clustering	320	247	77.2	[72.3, 81.4]	+6.16	$p < 0.001$
Hierarchical	Clustering	320	245	76.6	[71.6, 80.9]	+5.98	$p < 0.001$
DBSCAN_Hybrid	Hybrid	320	245	76.6	[71.6, 80.9]	+5.98	$p < 0.001$
KMeans	Clustering	320	238	74.4	[69.3, 78.8]	+5.36	$p < 0.001$
SCIZOR	Legacy	320	181	56.6	[51.1, 62.0]	+0.64	$p = 0.522$
General	General	320	173	54.1	[48.6, 59.4]	Baseline	Baseline
RuleBased_General	General	320	128	40.0	[34.8, 45.5]	−3.56	$p < 0.001$
BehaviorRetrieval	Legacy	320	118	36.9	[31.8, 42.3]	−4.37	$p < 0.001$
CUPID	Legacy	320	14	4.4	[2.6, 7.2]	−13.82	$p < 0.001$

Table A2 details the success rates broken down by both the trajectory filtering (Data Selection) and the imitation learning architecture.

Table A2. Success rate by data selection and learning method with 95% CI.

Data Selection	Learning Method	n	SR (%)	95% CI
KMeans_Hybrid	BC_MLP	80	73.8	[63.2, 82.1]
KMeans_Hybrid	BC_RNN	80	92.5	[84.6, 96.5]
KMeans_Hybrid	BC_RNN_Dropout	80	97.5	[91.3, 99.3]
KMeans_Hybrid	BC_RNN_MDN	80	52.5	[41.7, 63.1]
Hierarchical_Hybrid	BC_MLP	80	66.2	[55.4, 75.7]
Hierarchical_Hybrid	BC_RNN	80	96.2	[89.5, 98.7]
Hierarchical_Hybrid	BC_RNN_Dropout	80	91.2	[83.0, 95.7]
Hierarchical_Hybrid	BC_RNN_MDN	80	60.0	[49.0, 70.0]
DBSCAN	BC_MLP	80	71.2	[60.5, 80.0]
DBSCAN	BC_RNN	80	95.0	[87.8, 98.0]
DBSCAN	BC_RNN_Dropout	80	93.8	[86.2, 97.3]
DBSCAN	BC_RNN_MDN	80	48.8	[38.1, 59.5]
Hierarchical	BC_MLP	80	75.0	[64.5, 83.2]
Hierarchical	BC_RNN	80	92.5	[84.6, 96.5]
Hierarchical	BC_RNN_Dropout	80	92.5	[84.6, 96.5]
Hierarchical	BC_RNN_MDN	80	46.2	[35.7, 57.1]
DBSCAN_Hybrid	BC_MLP	80	73.8	[63.2, 82.1]
DBSCAN_Hybrid	BC_RNN	80	96.2	[89.5, 98.7]
DBSCAN_Hybrid	BC_RNN_Dropout	80	95.0	[87.8, 98.0]
DBSCAN_Hybrid	BC_RNN_MDN	80	41.2	[31.1, 52.2]
KMeans	BC_MLP	80	65.0	[54.1, 74.5]
KMeans	BC_RNN	80	92.5	[84.6, 96.5]
KMeans	BC_RNN_Dropout	80	97.5	[91.3, 99.3]
KMeans	BC_RNN_MDN	80	42.5	[32.3, 53.4]
SCIZOR	BC_MLP	80	52.5	[41.7, 63.1]
SCIZOR	BC_RNN	80	58.8	[47.8, 68.9]
SCIZOR	BC_RNN_Dropout	80	40.0	[30.0, 51.0]
SCIZOR	BC_RNN_MDN	80	75.0	[64.5, 83.2]
General	BC_MLP	80	56.2	[45.3, 66.6]
General	BC_RNN	80	82.5	[72.7, 89.3]
General	BC_RNN_Dropout	80	22.5	[14.7, 32.8]
General	BC_RNN_MDN	80	55.0	[44.1, 65.4]
RuleBased_General	BC_MLP	80	57.5	[46.6, 67.7]
RuleBased_General	BC_RNN	80	63.7	[52.8, 73.4]
RuleBased_General	BC_RNN_Dropout	80	8.8	[4.3, 17.0]
RuleBased_General	BC_RNN_MDN	80	30.0	[21.1, 40.8]
BehaviorRetrieval	BC_MLP	80	28.7	[20.0, 39.5]
BehaviorRetrieval	BC_RNN	80	46.2	[35.7, 57.1]
BehaviorRetrieval	BC_RNN_Dropout	80	45.0	[34.6, 55.9]
BehaviorRetrieval	BC_RNN_MDN	80	27.5	[18.9, 38.1]
CUPID	BC_MLP	80	11.2	[6.0, 20.0]
CUPID	BC_RNN	80	0.0	[0.0, 4.6]
CUPID	BC_RNN_Dropout	80	5.0	[2.0, 12.2]
CUPID	BC_RNN_MDN	80	1.2	[0.2, 6.7]

Appendix A.2. Task Robustness and Distance-Based Analysis

To evaluate the robustness of the trajectory filtering across different task difficulties, Table A3 presents success rates categorized by target distance ranges from the initial position.

Table A3. Success rate by distance range with 95% CI and p-value vs. General.

Data Selection	Dist (cm)	n	SR (%)	95% CI	p vs. Gen
KMeans_Hybrid	0	20	90.0	[69.9, 97.2]	$p = 0.002$
KMeans_Hybrid	1	60	88.3	[77.8, 94.2]	$p < 0.001$
KMeans_Hybrid	2	60	81.7	[70.1, 89.4]	$p < 0.001$
KMeans_Hybrid	3	60	73.3	[61.0, 82.9]	$p = 0.036$
KMeans_Hybrid	4	60	86.7	[75.8, 93.1]	$p = 0.006$
KMeans_Hybrid	5	60	61.7	[49.0, 72.9]	$p = 0.198$
Hierarchical_Hybrid	0	20	70.0	[48.1, 85.5]	$p = 0.110$
Hierarchical_Hybrid	1	60	80.0	[68.2, 88.2]	$p = 0.001$
Hierarchical_Hybrid	2	60	100.0	[94.0, 100.0]	$p < 0.001$
Hierarchical_Hybrid	3	60	88.3	[77.8, 94.2]	$p < 0.001$
Hierarchical_Hybrid	4	60	70.0	[57.5, 80.1]	$p = 0.559$
Hierarchical_Hybrid	5	60	56.7	[44.1, 68.4]	$p = 0.464$
DBSCAN	0	20	85.0	[64.0, 94.8]	$p = 0.008$
DBSCAN	1	60	86.7	[75.8, 93.1]	$p < 0.001$
DBSCAN	2	60	85.0	[73.9, 91.9]	$p < 0.001$
DBSCAN	3	60	86.7	[75.8, 93.1]	$p < 0.001$
DBSCAN	4	60	73.3	[61.0, 82.9]	$p = 0.232$
DBSCAN	5	60	50.0	[37.9, 62.1]	$p = 0.739$
Hierarchical	0	20	85.0	[64.0, 94.8]	$p = 0.008$
Hierarchical	1	60	93.3	[83.8, 97.4]	$p < 0.001$
Hierarchical	2	60	91.7	[82.0, 96.4]	$p < 0.001$
Hierarchical	3	60	83.3	[72.0, 90.6]	$p < 0.001$
Hierarchical	4	60	68.3	[55.8, 78.7]	$p = 0.780$
Hierarchical	5	60	46.7	[34.6, 59.1]	$p = 0.043$
DBSCAN_Hybrid	0	20	90.0	[69.9, 97.2]	$p = 0.002$
DBSCAN_Hybrid	1	60	90.0	[80.1, 95.3]	$p < 0.001$
DBSCAN_Hybrid	2	60	70.0	[57.5, 80.1]	$p = 0.001$
DBSCAN_Hybrid	3	60	85.0	[73.9, 91.9]	$p < 0.001$
DBSCAN_Hybrid	4	60	51.7	[39.5, 63.7]	$p = 0.613$
DBSCAN_Hybrid	5	60	41.7	[30.0, 54.3]	$p = 0.002$
KMeans	0	20	80.0	[58.7, 91.9]	$p = 0.025$
KMeans	1	60	81.7	[70.1, 89.4]	$p < 0.001$
KMeans	2	60	70.0	[57.5, 80.1]	$p = 0.001$
KMeans	3	60	80.0	[68.2, 88.2]	$p = 0.001$
KMeans	4	60	61.7	[49.0, 72.9]	$p = 0.198$
KMeans	5	60	75.0	[62.8, 84.2]	$p = 0.100$
SCIZOR	0	20	60.0	[38.7, 78.5]	$p = 0.204$
SCIZOR	1	60	53.3	[40.9, 65.4]	$p = 0.830$
SCIZOR	2	60	56.7	[44.1, 68.4]	$p = 0.464$
SCIZOR	3	60	45.0	[33.1, 57.5]	$p = 0.465$
SCIZOR	4	60	46.7	[34.6, 59.1]	$p = 0.043$
SCIZOR	5	60	66.7	[54.2, 77.3]	$p = 0.064$
General	0	20	45.0	[25.8, 65.8]	Baseline
General	1	60	51.7	[39.5, 63.7]	Baseline
General	2	60	53.3	[40.9, 65.4]	Baseline
General	3	60	50.0	[37.9, 62.1]	Baseline
General	4	60	60.0	[47.4, 71.5]	Baseline
General	5	60	53.3	[40.9, 65.4]	Baseline

Table A3. *Cont.*

Data Selection	Dist (cm)	n	SR (%)	95% CI	*p* vs. Gen
RuleBased_General	0	20	30.0	[13.8, 53.8]	$p = 0.300$
RuleBased_General	1	60	36.7	[25.6, 49.3]	$p = 0.098$
RuleBased_General	2	60	45.0	[33.1, 57.5]	$p = 0.465$
RuleBased_General	3	60	36.7	[25.6, 49.3]	$p = 0.044$
RuleBased_General	4	60	46.7	[34.6, 59.1]	$p = 0.043$
RuleBased_General	5	60	33.3	[22.7, 45.9]	$p = 0.064$
BehaviorRetrieval	0	20	40.0	[21.9, 61.3]	$p = 0.749$
BehaviorRetrieval	1	60	36.7	[25.6, 49.3]	$p = 0.098$
BehaviorRetrieval	2	60	31.7	[21.3, 44.2]	$p = 0.026$
BehaviorRetrieval	3	60	38.3	[27.1, 51.0]	$p = 0.067$
BehaviorRetrieval	4	60	36.7	[25.6, 49.3]	$p = 0.002$
BehaviorRetrieval	5	60	40.0	[28.6, 52.6]	$p = 0.271$
CUPID	0	20	5.0	[0.9, 23.6]	$p = 0.003$
CUPID	1	60	6.7	[2.6, 15.9]	$p < 0.001$
CUPID	2	60	5.0	[1.7, 13.7]	$p < 0.001$
CUPID	3	60	3.3	[0.9, 11.4]	$p < 0.001$
CUPID	4	60	5.0	[1.7, 13.7]	$p < 0.001$
CUPID	5	60	1.7	[0.3, 8.9]	$p < 0.001$

Appendix A.3. Trajectory Quality and Efficiency Analysis

The following analysis focuses on the qualitative aspects of the generated trajectories, including Average Final Distance (FD), Average Trajectory Length (PL), and Average Joint Movement (JM). Figure A2 and Table A4 demonstrate that the hybrid filtering methods significantly enhance trajectory smoothness and precision, where blue, green, red, and orange represent Clustering, Hybrid, Legacy, and General methods, respectively.

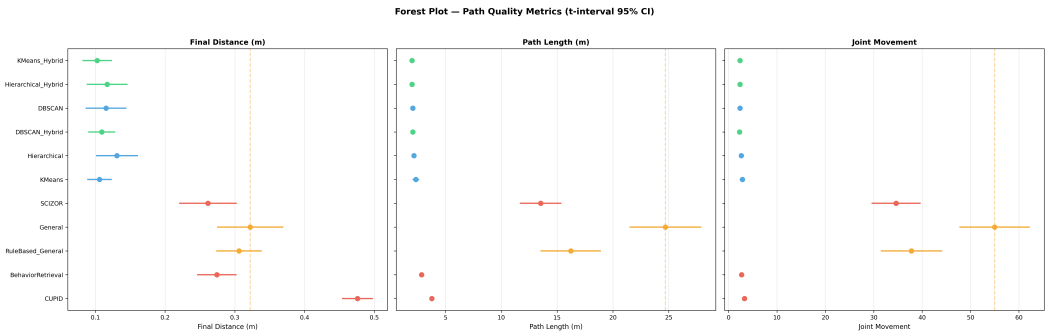

Figure A2. Forest plot of trajectory quality metrics (t-interval 95% CI) vs. General baseline (dashed line).

Table A4. Trajectory quality metrics with 95% CI and *t*-test vs. General baseline.

Data Selection	Group	n	Avg FD (m)	FD CI	FD p	Avg PL (m)	PL CI	PL p	Avg JM	JM CI	JM p
KMeans_Hybrid	Hybrid	320	0.103	[0.081, 0.124]	$p < 0.001$	2.003	[1.895, 2.111]	$p < 0.001$	2.356	[2.282, 2.430]	$p < 0.001$
Hier_Hybrid	Hybrid	320	0.117	[0.088, 0.146]	$p < 0.001$	2.007	[1.898, 2.116]	$p < 0.001$	2.364	[2.289, 2.440]	$p < 0.001$
DBSCAN	Cluster	320	0.115	[0.086, 0.145]	$p < 0.001$	2.078	[1.966, 2.190]	$p < 0.001$	2.361	[2.279, 2.442]	$p < 0.001$
Hierarchical	Cluster	320	0.131	[0.101, 0.161]	$p < 0.001$	2.177	[1.990, 2.363]	$p < 0.001$	2.620	[2.271, 2.969]	$p < 0.001$
DBSCAN_Hybrid	Hybrid	320	0.109	[0.090, 0.129]	$p < 0.001$	2.074	[1.966, 2.182]	$p < 0.001$	2.274	[2.193, 2.355]	$p < 0.001$
KMeans	Cluster	320	0.106	[0.088, 0.123]	$p < 0.001$	2.350	[2.049, 2.651]	$p < 0.001$	2.852	[2.301, 3.403]	$p < 0.001$
SCIZOR	Legacy	320	0.261	[0.221, 0.301]	$p = 0.058$	13.520	[11.668, 15.372]	$p < 0.001$	34.615	[29.560, 39.670]	$p < 0.001$
General	General	320	0.322	[0.274, 0.370]	Baseline	24.700	[21.487, 27.913]	Baseline	54.967	[47.709, 62.225]	Baseline
RuleBased_G	General	320	0.306	[0.273, 0.339]	$p = 0.586$	16.222	[13.513, 18.931]	$p < 0.001$	37.789	[31.450, 44.129]	$p < 0.001$
BehaviorRet.	Legacy	320	0.274	[0.246, 0.303]	$p = 0.091$	2.867	[2.731, 3.003]	$p < 0.001$	2.684	[2.587, 2.781]	$p < 0.001$
CUPID	Legacy	320	0.476	[0.454, 0.498]	$p < 0.001$	3.784	[3.724, 3.844]	$p < 0.001$	3.301	[3.219, 3.383]	$p < 0.001$

Computational efficiency is analyzed in Table A5, detailing the selection time and the subsequent training cost reductions.

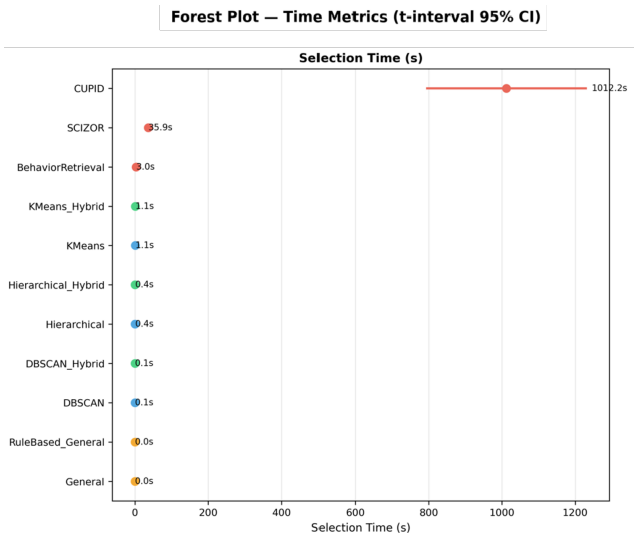

Figure A3. Forest plot of time metrics (t-interval 95% CI) vs. General baseline (dashed line).

Table A5. Data selection time with 95% CI and *t*-test vs. General baseline.

Data Selection	Group	Sel Time (s)	Sel CI	Sel p	n
General	General	0.00	[0.00, 0.00]	Baseline	18
BehaviorRet.	Legacy	2.97	[2.96, 2.98]	$p < 0.001$	18
CUPID	Legacy	1012.19	[792.6, 1231.8]	$p < 0.001$	18
DBSCAN	Cluster	0.07	[0.07, 0.08]	$p < 0.001$	18
DBSCAN_Hybrid	Hybrid	0.09	[0.09, 0.09]	$p < 0.001$	18
Hierarchical	Cluster	0.38	[0.38, 0.38]	$p < 0.001$	18
Hier_Hybrid	Hybrid	0.40	[0.39, 0.40]	$p < 0.001$	18
KMeans	Cluster	1.06	[1.03, 1.08]	$p < 0.001$	18
KMeans_Hybrid	Hybrid	1.07	[1.05, 1.08]	$p < 0.001$	18
RuleBased_G	General	0.00	[0.00, 0.00]	N/A	18
SCIZOR	Legacy	35.88	[35.81, 35.95]	$p < 0.001$	18

References

1. Waseem, S.; Adnan, M.; Iqbal, M.S.; Amin, A.A.; Shah, A.; Tariq, M. From classical to intelligent control: Evolving trends in robotic manipulator technology. *Comput. Electr. Eng.* **2025**, *127*, 110559. [CrossRef]
2. Li, B.; Li, X.; Gao, H.; Wang, F.-Y. Advances in Flexible Robotic Manipulator Systems—Part I: Overview and Dynamics Modeling Methods. *IEEE/ASME Trans. Mechatron.* **2024**, *29*, 1100–1110. [CrossRef]
3. Merckaert, K.; Convens, B.; Nicotra, M.M.; Vanderborght, B. Real-time constraint-based planning and control of robotic manipulators for safe human–robot collaboration. *Robot. Comput.-Integr. Manuf.* **2024**, *87*, 102711. [CrossRef]
4. Orthey, A.; Chamzas, C.; Kavraki, L.E. Sampling-Based Motion Planning: A Comparative Review. *Annu. Rev. Control Robot. Auton. Syst.* **2024**, *7*, 285–310. [CrossRef]
5. Karaman, S.; Frazzoli, E. Sampling-Based Algorithms for Optimal Motion Planning. *Int. J. Robot. Res.* **2011**, *30*, 846–894. [CrossRef]
6. Qureshi, A.H.; Miao, Y.; Simeonov, A.; Yip, M.C. Motion Planning Networks: Bridging the Gap Between Learning-Based and Classical Motion Planners. *IEEE Trans. Robot.* **2021**, *37*, 48–66. [CrossRef]
7. De Carvalho, G.P.; Sawanobori, T.; Horii, T. Data-Driven Motion Planning: A Survey on Deep Neural Networks, Reinforcement Learning, and Large Language Model Approaches. *IEEE Access* **2025**, *13*, 52195–52245. [CrossRef]
8. Delgado, J.M.D.; Oyedele, L. Robotics in construction: A critical review of the reinforcement learning and imitation learning paradigms. *Adv. Eng. Inform.* **2022**, *54*, 101787. [CrossRef]
9. Han, D.; Mulyana, B.; Stankovic, V.; Cheng, S. A survey on deep reinforcement learning algorithms for robotic manipulation. *Sensors* **2023**, *23*, 3762. [CrossRef]
10. Correia, A.; Alexandre, L.A. A survey of demonstration learning. *Robot. Auton. Syst.* **2024**, *182*, 104812. [CrossRef]
11. Mohaghegh, N.; Wang, H.; Yazdani, A. Sim2Real Transfer of Imitation Learning of Motion Control for Car-like Mobile Robots Using Digital Twin Testbed. *Robotics* **2025**, *14*, 180. [CrossRef]
12. Sandakalum, T.; Ang, M.H. Motion Planning for Mobile Manipulators—A Systematic Review. *Machines* **2022**, *10*, 97. [CrossRef]

13. Tobin, J.; Fong, R.; Ray, A.; Schneider, J.; Zaremba, W.; Abbeel, P. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. In *Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*; IEEE: New York, NY, USA, 2017; pp. 23–30.
14. Luo, S.; Schomaker, L. Reinforcement learning in robotic motion planning by combined experience-based planning and self-imitation learning. *Robot. Auton. Syst.* **2023**, *170*, 104545. [CrossRef]
15. Barekatain, A.; Habibi, H.; Voos, H. A Practical Roadmap to Learning from Demonstration for Robotic Manipulators in Manufacturing. *Robotics* **2024**, *13*, 100. [CrossRef]
16. Osa, T.; Pajarinen, J.; Neumann, G.; Bagnell, J.A.; Abbeel, P.; Peters, J. An Algorithmic Perspective on Imitation Learning. *Found. Trends Robot.* **2018**, *7*, 1–179. [CrossRef]
17. Schaal, S. Learning from Demonstration. *Adv. Neural Inf. Process. Syst.* **1996**, *9*, 1041–1046.
18. O'Neill, A.; Rehman, A.; Maddukuri, A.; Gupta, A.; Padalkar, A.; Lee, A. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. In *Proceedings of the 2024 IEEE International Conference on Robotics and Automation (ICRA)*; IEEE: New York, NY, USA, 2024; pp. 6892–6903.
19. Nahavandi, S.; Alizadehsani, R.; Nahavandi, D.; Lim, C.P.; Kelly, K.; Bello, F. Machine learning meets advanced robotic manipulation. *Inf. Fusion* **2024**, *105*, 102221. [CrossRef]
20. Gao, J.; Xie, A.; Xiao, T.; Finn, C.; Sadigh, D. Efficient Data Collection for Robotic Manipulation via Compositional Generalization. *arXiv* **2024**, arXiv:2403.05110. [CrossRef]
21. Chen, H.; Zhu, C.; Liu, S.; Li, Y.; Driggs-Campbell, K. Tool-as-Interface: Learning Robot Policies from Observing Human Tool Use. *arXiv* **2025**, arXiv:2504.04612.
22. Du, M.; Nair, S.; Sadigh, D.; Finn, C. Behavior Retrieval: Few-Shot Imitation Learning by Querying Unlabeled Datasets. In Proceedings of the Robotics: Science and Systems 2023, Daegu, Republic of Korea, 10–14 July 2023.
23. Agia, C.; Sinha, R.; Yang, J.; Antonova, R.; Pavone, M.; Nishimura, H.; Itkina, M.; Bohg, J. CUPID: Curating Data Your Robot Loves with Influence Functions. *arXiv* **2025**, arXiv:2506.19121. [CrossRef]
24. Zhang, Y.; Xie, Y.; Liu, H.; Shah, R.; Wan, M.; Fan, L.; Zhu, Y. SCIZOR: A Self-Supervised Approach to Data Curation for Large-Scale Imitation Learning. *arXiv* **2025**, arXiv:2505.22626.
25. Zhu, W.; Guo, X.; Owaki, D.; Kutsuzawa, K.; Hayashibe, M. A survey of sim-to-real transfer techniques applied to reinforcement learning for bioinspired robots. *IEEE Trans. Neural Netw. Learn. Syst.* **2021**, *34*, 3444–3459. [CrossRef]
26. Hussein, A.; Gaber, M.M.; Elyan, E.; Jayne, C. Imitation Learning: A Survey of Learning Methods. *ACM Comput. Surv.* **2017**, *50*, 1–35. [CrossRef]
27. Zare, M.; Kebria, P.M.; Khosravi, A.; Nahavandi, S. A Survey of Imitation Learning: Algorithms, Recent Developments, and Challenges. *IEEE Trans. Cybern.* **2024**, *54*, 7173–7186. [CrossRef]
28. Ozalp, R.; Ucar, A.; Guzelis, C. Advancements in Deep Reinforcement Learning and Inverse Reinforcement Learning for Robotic Manipulation. *IEEE Access* **2024**, *12*, 51840–51858. [CrossRef]
29. Ross, S.; Gordon, G.; Bagnell, D. A Reduction of Imitation Learning to No-Regret Online Learning. In *JMLR Workshop and Conference Proceedings, Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS) 2011, Fort Lauderdale, FL, USA, 11–13 April 2011*; Proceedings of Machine Learning Research: Cambridge, MA, USA, 2011; pp. 627–635.
30. Ho, J.; Ermon, S. Generative Adversarial Imitation Learning. In Proceedings of the Advances in Neural Information Processing Systems, Barcelona, Spain, 5–10 December 2016.
31. Fu, J.; Luo, K.; Levine, S. Learning Robust Rewards with Adversarial Inverse Reinforcement Learning. In Proceedings of the International Conference on Learning Representations 2018, Vancouver, BC, Canada, 30 April–3 May 2018.
32. Cho, K.; van Merriënboer, B.; Gulcehre, C.; Bahdanau, D.; Bougares, F.; Schwenk, H.; Bengio, Y. Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP) 2014* ; Association for Computational Linguistics: Doha, Qatar, 2014; pp. 1724–1734.
33. Chen, L.; Lu, K.; Rajeswaran, A.; Lee, K.; Grover, A.; Laskin, M.; Abbeel, P.; Srinivas, A.; Mordatch, I. Decision Transformer: Reinforcement Learning via Sequence Modeling. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 15084–15097.
34. Srivastava, N.; Hinton, G.; Krizhevsky, A.; Sutskever, I.; Salakhutdinov, R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *J. Mach. Learn. Res.* **2014**, *15*, 1929–1958.
35. Gal, Y.; Ghahramani, Z. A Theoretically Grounded Application of Dropout in Recurrent Neural Networks. *Adv. Neural Inf. Process. Syst.* **2016**, *29, 1019–1027.*
36. Bishop, C.M. *Mixture Density Networks*; Technical Report NCRG/94/004; Aston University: Birmingham, UK, 1994.
37. Finn, C.; Yu, T.; Zhang, T.; Abbeel, P.; Levine, S. One-Shot Visual Imitation Learning via Meta-Learning. In Proceedings of the Conference on Robot Learning (CoRL), Mountain View, CA, USA, 13–15 November 2017; pp. 357–368.
38. Duan, Y.; Andrychowicz, M.; Stadie, B.; Ho, J.; Schneider, J.; Sutskever, I.; Zaremba, W. One-Shot Imitation Learning. In Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA, 4–9 December 2017.

39. Szep, M.; Lauenburg, L.; Farkas, K.; Su, X.; Zang, C. Reinforcement Learning for Solving Robotic Reaching Tasks in the Neurorobotics Platform. *arXiv* **2022**, arXiv:2210.17138. [CrossRef]
40. Wang, T.; Peng, X.; Lei, X.; Wang, H.; Jin, Y. Knowledge-assisted evolutionary task scheduling for hierarchical multiagent systems with transferable surrogates. *Swarm Evol. Comput.* **2025**, *98*, 102107. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.